

```
# Plan de Aseguramiento de la Calidad del Software (SQAP)
**Proyecto**: Visualizador de Marcos (Simulador 3D Enmarcame)
**Versión**: 1.0
**Fecha**: 2026-05-18
**Estado**: #estado/finalizado
```

```
---
```

1. Propósito

El presente documento define el Plan de Aseguramiento de la Calidad (SQAP) para el proyecto "Visualizador de Marcos". Su objetivo principal es garantizar que los procesos de desarrollo y los artefactos producidos cumplan con los estándares de calidad establecidos por la organización XookTech, fundamentados en el marco teórico de Daniel Galin y el estándar IEEE 730.

Este plan se enfoca prioritariamente en la mejora de dos procesos críticos identificados previamente:

1. **PROC-02: Especificación de Requerimientos** (para eliminar la ambigüedad).
2. **PROC-05: Plan de Pruebas** (para asegurar la verificación de los requerimientos recuperados).

```
---
```

2. Documentos de Referencia

- **Galín, D. (2004)**: Software Quality Assurance: From theory to implementation.
- **IEEE Std 730-2014**: Standard for Software Quality Assurance Processes.
- **PROC-00**: Gobernanza del SGC - XookTech.
- **STD-01**: Estándar de Estructuración de Procesos ETVX.
- **STD-02**: Convenciones de Nomenclatura y Tags.

```
---
```

3. Gestión (Administración de la Calidad)

3.1 Estructura Organizacional y Roles

El equipo de XookTech opera bajo una estructura de independencia funcional para asegurar que el control de calidad no se vea comprometido por los plazos de desarrollo.

Rol	Responsabilidad Principal	Tarea SQA Asociada
Analista de Gobernanza	Integridad del SGC y Vault	Auditoría de procesos y cumplimiento normativo.
Analista de Requerimientos	Gestión de necesidades (IEEE 830)	Verificación de completitud y trazabilidad de REQs.
Líder de Desarrollo	Implementación técnica y Despliegue	Auto-inspección de código y revisión de pares.
Analista de Control	Gestión de Cambios (CR)	Auditoría de impacto y control de versiones.
Analista de Verificación	Diseño y ejecución de Pruebas	Validación de criterios de aceptación.

3.2 Tareas de Aseguramiento

- **Revisiones Técnicas Formales (FTR)**: Aplicadas a los documentos de requerimientos.

- **Auditorías de Proceso**: Verificación del cumplimiento del rigor ETVX en cada fase.
- **Monitoreo de Métricas**: Seguimiento de la densidad de defectos y cobertura de pruebas.

4. Documentación

Se requiere que cada fase del ciclo de vida genere los siguientes artefactos obligatorios bajo el estándar ETVX:

- **Fase de Requisitos**: REQ (Especificación), CL-02 (Checklist de Verificación).
- **Fase de Pruebas**: PLAN-CP (Plan Maestro), REG-CP (Registro de Resultados).
- **Fase de Control**: HALLAZGO (Discrepancias), CR (Solicitud de Cambio).

5. Estándares, Prácticas y Convenciones

- **Lenguaje**: Python 3.x con librerías OpenCV y Pillow.
- **Documentación**: Markdown en Obsidian con Frontmatter YAML obligatorio.
- **Nomenclatura**: Prefijos estandarizados (`PROC-`, `REQ-`, `CP-`).
- **Rigor Metodológico**: Toda tarea debe definirse como **Task** dentro de una matriz de Entradas y Salidas.

6. Mejora de Procesos (Detalle de los 2 Procesos Prioritarios)

6.1 Mejora en PROC-02: Especificación de Requerimientos

Problema original: Los requisitos se redactaban de forma "al aire", sin medidas técnicas (ej. "que el marco se vea bien").

Especificación de la Mejora:

- **Técnica de Desglose**: Se implementa el uso de **Atributos de Calidad** (IEEE 830). Cada requerimiento ahora debe incluir una métrica de éxito. Por ejemplo, el requerimiento de "Visualización 3D" ahora se divide en:
 - Precisión geométrica (tolerancia de 2mm en uniones).
 - Tiempo de respuesta (renderizado en menos de 2 segundos).
- **Validación Matemática**: El Analista de Requerimientos debe verificar que cada funcionalidad tenga una entrada y una salida lógica definida antes de pasar a desarrollo.
- **Resultado Esperado**: Reducción del 40% en las dudas de programación al tener reglas de negocio claras desde el inicio.

6.2 Mejora en PROC-05: Plan de Pruebas

Problema original: Se probaba solo "lo que funcionaba", ignorando los errores que el usuario podría cometer.

Especificación de la Mejora:

- **Matriz de Trazabilidad**: Se obliga a que cada prueba esté amarrada a un requerimiento del punto 6.1. Si no hay requerimiento, no hay prueba; si hay requerimiento, debe haber al menos dos pruebas (una de éxito y una de error).
- **Pruebas de "Caja Negra"**: El Analista de Verificación simula ser un usuario inexperto para intentar "romper" el simulador (ej. subir archivos de imagen corruptos o medidas negativas).
- **Evidencia Técnica**: Se capturan los errores detectados para que el programador sepa exactamente en qué línea de código falló el sistema.

7. Revisiones y Evaluación del Progreso

En lugar de trámites burocráticos, el equipo realiza sesiones de revisión directa donde se "cuentan" y analizan las desviaciones:

1. ****Conteo de Errores en Requisitos****: Al finalizar el PROC-02, el equipo se reúne para contar cuántas veces se usaron palabras ambiguas (como "rápido", "bonito", "eficiente"). Si el conteo es mayor a 0, el documento se regresa a corrección inmediata.
2. ****Verificación de Pasos (ETVX)****: Se revisa que cada proceso tenga sus entradas y salidas conectadas. Por ejemplo, se cuenta que cada diagrama de diseño tenga su correspondiente módulo de código. Si falta una conexión, se registra como una "tarea pendiente de calidad".
3. ****Evaluación de Pruebas****: Se cuenta el porcentaje de casos de prueba que pasaron a la primera. Un porcentaje bajo indica que el proceso de desarrollo (PROC-04) necesita reforzarse con mejores estándares de codificación.

8. Acciones Correctivas

Cuando se detecta que algo no cuadra con el plan:

- Se identifica la causa (¿falta de capacitación? ¿malas herramientas?).
- El líder del equipo da instrucciones directas para corregir el error en un plazo no mayor a 24 horas.
- Se actualiza el manual de procesos para que el mismo error no vuelva a ocurrir en el siguiente módulo.

9. Herramientas, Técnicas y Metodologías

- ****Obsidian****: Gestor de la Base de Conocimientos de Calidad (Vault).
- ****GitHub****: Control de versiones del código fuente y documentación.
- ****ETVX (Entry-Task-Validation-Exit)****: Metodología para la definición de procesos.
- ****PDCA (Plan-Do-Check-Act)****: Ciclo para la mejora continua del SGC.

10. Gestión de Riesgos

- ****Riesgo****: Incompletitud de la documentación por presión de tiempo.
- ****Mitigación****: Uso obligatorio de plantillas (`TEMPLATE-REQ`, `TEMPLATE-CP`) para acelerar la creación de artefactos sin sacrificar el estándar.

****Elaborado por****: Gemini CLI (Analista SQA Virtual)

****Aprobado por****: Equipo de Proyecto Visualizador de Marcos